

Spanning Tree

1. A spanning tree is a subgraph of a connected graph.
2. It includes all the vertices of the original graph.
3. It contains exactly $V - 1$ edges for V vertices.
4. It has no cycles (acyclic).
5. It is always connected.
6. Multiple different spanning trees are possible for the same graph.
7. It represents the simplest possible structure connecting all nodes.

Minimum Spanning Tree (MST)

1. A minimum spanning tree is a spanning tree with minimum total edge weight.
2. It connects all vertices without forming any cycles.
3. It also contains exactly $V - 1$ edges.
4. Among all spanning trees, MST uses the least-cost combination of edges.
5. MST is used in network design, routing, and cable optimization.
6. Algorithms like Prim's and Kruskal's are used to find MST.
7. MST is unique if all edge weights are distinct; otherwise, multiple MSTs may exist.

KRUSKAL'S ALGORITHM

1. Definition

Kruskal's Algorithm is a Greedy algorithm used to find a Minimum Spanning Tree (MST) of a connected, weighted graph.

It selects edges in ascending order of weight, and uses Union-Find to avoid cycles.

2. Key Properties

- Works for weighted, undirected graphs
- Uses sorting + DSU (Union-Find)
- Always produces a minimum spanning tree
- Time complexity:
- Sorting edges $\rightarrow O(E \log E)$
- Union-Find operations $\rightarrow$ almost $O(1)$

3. When Kruskal is best?

- Graph is sparse (few edges)
- Edge list is available
- Want MST quickly with simpler implementation

Kruskal's Algorithm

```
// Algo Kruskal(G)
// Input  : A connected weighted graph G(V, E)
// Output: Min Spanning Tree (MST)
```

Start

```
1. List all the edges of the graph.
2. Sort(E)                                //sort edges in increasing order
3. MST = { }                             // initialize MST as empty

4. For each edge in Sort(E):
    if (no cycle):
        MST.push(E)    // include the edge
    else
        skip(E)        // cycle → skip
5. Continue until MST has (V - 1) edges.
```

Stop

Kruskal's Algorithm (Using Union–Find)

```
// Algo Kruskal_UnionFind(G)

// Input: Graph G = (V, E) with edge weights

// Output: Minimum Spanning Tree (MST)

1. Create a set for each vertex using MakeSet(v)
2. Sort all edges in non-decreasing order of their weights
3. MST = empty
4. For each edge (u, v, w) in sorted order:
```

```
        if Find(u) ≠ Find(v):                // check if u and v
belong to different sets
```

```
        Add edge (u, v, w) to MST
```

```
        Union(u, v)                        // merge the sets
```

```
5. Return MST
```

Prim's Algorithm

1. Definition

Prim's Algorithm is a **greedy algorithm** used to find the **Minimum Spanning Tree (MST)** of a connected, weighted graph.

It starts from **any one vertex** and expands the MST by always selecting the **minimum-weight edge** that connects a visited vertex to an unvisited vertex.

2. When to Use Prim's

- Graph is dense
- You want to grow MST from a starting node
- Adjacency matrix works well
- Better performance with priority queue (Min-Heap)

Prim's Algorithm

```
// Algo Prim(G)
// Input  : A connected weighted graph G(V, E)
// Output: Min Spanning Tree (MST)
```

Start

1. Select any start vertex.
2. Mark it as visited.
3. $MST = \{ \}$ // initialize MST as empty
4. While MST has fewer than $(V - 1)$ edges:
 Choose the minimum-weight edge
 that connects visited $\rightarrow$ unvisited vertex.

 Add that edge to MST.
 Mark the new vertex as visited.
5. Continue until MST has $(V - 1)$ edges.

Stop

Difference Between Prim's and Kruskal's Algorithm

Feature / Point	Prim's Algorithm	Kruskal's Algorithm
Approach	Grows MST from a starting vertex by adding the minimum outgoing edge	Builds MST by selecting the globally minimum edges
Type	Vertex-based greedy	Edge-based greedy
Cycle Handling	Does not check for cycle explicitly	Uses Union-Find (DSU) to avoid cycles
Data Structure Used	Priority Queue / Min-Heap	Sorting + Union-Find
Works on	Works only on connected graphs	Works on connected & disconnected graphs (forms forest)

Where edges are checked	From the current tree to outside vertices	From entire edge list
Best For	Dense graphs	Sparse graphs
Time Complexity	$O(E \log V)$ with Min-Heap	$O(E \log E)$ with sorted edges
Starting Point	Must choose a source vertex	No starting vertex required
Nature	Incremental – expands step-by-step	Global – picks smallest edges first
MST Type	Always produces single MST from the starting vertex	Can produce forest for disconnected graph